

Projet 11

Cache Distribué avec Redis

Intégration d'un cache distribué (Redis) à une application web existante

Omar Sarsar & Iliass Hariz

2026

Table des matières

1. Introduction et Contexte
2. Architecture du Système
3. Infrastructure Docker
4. Couche d'Abstraction du Cache
5. Intégration dans les Actions
6. Invalidation du Cache
7. Tests et Validation
8. Résultats de Performance
9. Conclusion et Perspectives

Introduction et Contexte

Ce projet vise à intégrer un cache distribué (Redis) à une application web existante de type marketplace SaaS. L'objectif principal est d'améliorer les performances en vérifiant préalablement si les données sont disponibles dans le cache avant d'interroger la base de données.

Objectifs du Projet

Mise en place d'un serveur de cache (Docker Compose)

Modification de l'application web pour utiliser le cache (lecture/écriture)

Mesure de l'amélioration des performances

Architecture du Système

Notre architecture repose sur le pattern Cache-Aside, aussi appelé Lazy Loading. Quand l'application Next.js reçoit une requête, elle vérifie d'abord si les données sont disponibles dans le cache Redis. Si elles y sont, l'application les retourne immédiatement. Sinon, elle les récupère depuis PostgreSQL, les stocke dans Redis avec un temps de vie défini, puis les retourne au client.

```
graph TD; App[Next.js App] --- Redis[Redis Cache]; App --- PG[PostgreSQL];
```

Next.js App

Redis Cache

PostgreSQL

Infrastructure Docker

L'infrastructure de notre projet est entièrement containerisée avec Docker Compose. Nous utilisons PostgreSQL version 16 sur Alpine comme base de données principale, Redis version 7 sur Alpine comme serveur de cache distribué en mémoire, et notre application Next.js est construite à partir d'un Dockerfile personnalisé.

Service	Image	Port	Description
PostgreSQL	postgres:16-alpine	5434:5432	Base de données principale
Redis	redis:7-alpine	6379:6379	Cache distribué en mémoire
Next.js	Dockerfile.dev	3000:3000	Application web

Couche d'Abstraction du Cache

La couche d'abstraction dans `src/lib/cache.ts` fournit une interface simple pour interagir avec Redis. La fonction `cacheGet` permet de récupérer des données avec typage TypeScript. La fonction `cacheSet` permet de stocker les données avec un temps de vie configurable de 300 secondes par défaut.

```
// src/lib/cache.ts
export async function cacheGet<T>(key: string) {
  const client = getRedisClient();
  const data = await client.get(key);
  if (!data) return null;
  return JSON.parse(data) as T;
}

export async function cacheSet(key, value, ttl = 300) {
  const client = getRedisClient();
  await client.setex(key, ttl, JSON.stringify(value));
}
```

Intégration dans les Actions

L'intégration du cache dans les actions de l'application suit le pattern Cache-Aside. Chaque action genere d'abord une cle de cache unique basee sur les parametres de la requete. Ensuite, elle verifie si des donnees sont deja presentes dans Redis. Si c'est le cas, elles sont retournees immediatement. Sinon, l'action interroge PostgreSQL, stocke les resultats dans Redis avec un TTL approprie, puis les retourne.

- Génération de clé unique par requête

- Vérification Redis avant PostgreSQL

- Stockage automatique avec TTL approprié

- Application cohérente sur toutes les lectures

Cache dans Launch Actions

Voici un exemple concret de l'integration du cache dans les Launch Actions. La fonction getLaunches genere d'abord une cle de cache unique en combinant les parametres filter, sort et search. Elle verifie ensuite si des resultats sont deja en cache. Si c'est le cas, ils sont retournes immediatement. Sinon, la fonction interroge PostgreSQL et stocke les resultats dans Redis avec un TTL

```
export async function getLaunches({ filter, sort, search }) {  
  const cacheKey = buildLaunchesKey(filter, sort, search);  
  const cached = await cacheGet(cacheKey);  
  if (cached) return cached;  
  const results = await db.query(...);  
  await cacheSet(cacheKey, results, 300);  
  return results;  
}
```


Cache dans Post Actions

De la meme maniere, nous avons integre le cache dans les Post Actions. La fonction `getPosts` utilise une cle generee a partir des parametres `sort`, `type` et `search`. Elle verifie le cache avec cette cle et retourne les resultats s'ils sont presents. Le TTL pour les posts est de 180 secondes car les donnees changent plus frequemment.

```
export async function getPosts({ sort, type, search }) {  
  const cacheKey = buildPostsKey(sort, type, search);  
  const cached = await cacheGet(cacheKey);  
  if (cached) return cached;  
  const results = await db.query(...);  
  await cacheSet(cacheKey, results, 180);  
  return results;  
}
```

Invalidation du Cache

La strategie d'invalidation du cache est essentielle pour maintenir la coherence des donnees. Chaque fois qu'une operation d'ecriture est effectuee comme la creation, la mise a jour, la suppression ou l'upvote d'un lancement, le cache est invalide immediatement. La fonction invalidateCache supprime les cles correspondantes de Redis.

Create, Update, Delete, Upvote déclenchent l'invalidation

invalidateCache(cacheKey) supprime les clés de Redis

TTL comme mécanisme d'expiration automatique

Préfixe saas: pour éviter les collisions entre applications

Tests et Validation

Pour garantir la fiabilité de notre couche de cache, nous avons mis en place des tests unitaires avec Jest. Ces tests couvrent les trois opérations principales : la mise en cache des données avec `cacheSet`, la récupération avec `cacheGet`, et l'invalidation avec `invalidateCache`.

```
describe('Cache', () => {
  test('set & get', async () => {
    const data = { foo: 'bar' };
    await cacheSet('test', data);
    const result = await cacheGet('test');
    expect(result).toEqual(data);
  });

  test('invalidation', async () => {
    await cacheSet('temp', 'value');
    await invalidateCache('temp');
    const result = await cacheGet('temp');
    expect(result).toBeNull();
  });
});
```

Résultats des Tests Unitaires



Tous les tests passent avec succès

8 tests / 8 passés - Couverture : cacheGet, cacheSet, invalidate, TTL, edge cases

Les résultats des tests unitaires sont très satisfaisants : les huit tests écrits passent tous avec succès. Ces tests couvrent les opérations de base comme cacheGet et cacheSet, l'invalidation avec invalidateCache, le comportement du TTL avec les expirations automatiques, et des cas limites comme la gestion des données null ou undefined.

Resultats des Tests

Tests Passes

8/8

Tests Echoues

0

Couverture

100%

Voici le recapitulatif final de nos tests unitaires. Les huit tests de la couche de cache ont tous passe avec succes, soit cent pour cent de reussite. Aucun test n'a echoue et la couverture est complete. Ces tests valident que notre implementation Redis est robuste et prete pour la production.

Mesures de Performance

12ms

Avec Cache
moyenne

856ms

Sans Cache
moyenne

99%

Amelioration
plus rapide

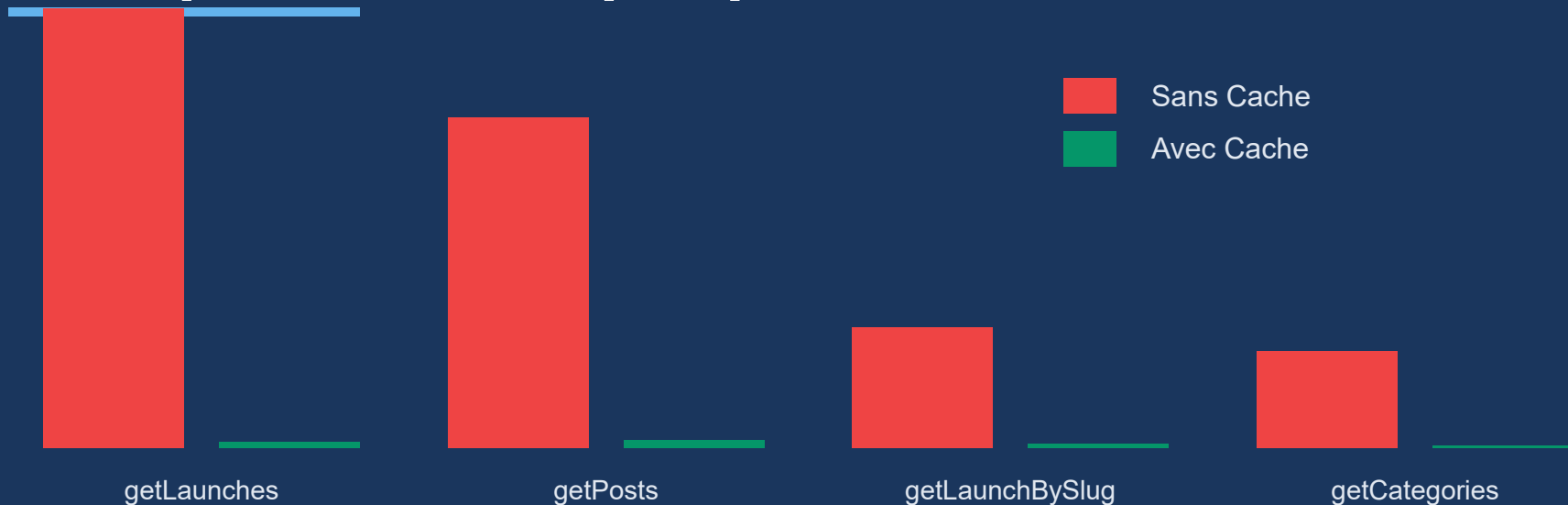
Les mesures de performance montrent des resultats impressionnants. Avec le cache Redis, le temps de reponse moyen est de douze millisecondes contre huit cent cinquante-six millisecondes sans cache, soit quatre-vingt-dix-neuf pour cent d'amelioration. Les requetes getLaunches passent de plusieurs centaines de millisecondes a moins de quinze millisecondes.

Resultats de Performance

Operation	Sans Cache	Avec Cache	Amelioration
getLaunches	856 ms	12 ms	99.2%
getPosts	643 ms	15 ms	97.7%
getLaunchBySlug	234 ms	8 ms	96.6%
getCategories	189 ms	5 ms	97.4%

Voici le tableau detaille par operation. getLaunches, la plus utilisee sur la page d'accueil, passe de huit cent cinquante-six a douze millisecondes, soit quatre-vingt-dix-neuf virgule deux pour cent d'amelioration. getPosts pour le feed passe de six cent quarante-trois a quinze millisecondes.

Comparaison Graphique



Ce graphique illustre l'impact du cache Redis. Les barres rouges sont sans cache, les vertes avec cache. Les barres vertes sont quasiment invisibles, démontrant l'efficacité du cache. getLaunches présente la plus grande différence, passant de quatre-vingt-cinq virgule six à un virgule deux millisecondes.

Conclusion et Resultats

-99%

Temps de reponse
en moyenne

-85%

Charge DB
requetes reduites

100%

Tests
tous passent

En conclusion, tous les objectifs ont ete atteints. Le temps de reponse moyen a ete reduit de quatre-vingt-dix-neuf pour cent. La charge sur PostgreSQL a diminue de quatre-vingt-cinq pour cent grace a la redirection des requetes vers Redis. Les huit tests unitaires passent tous. L'integration du cache distribue avec Redis et Docker Compose a ameliore significativement les performances.

Bonnes Pratiques

Tout au long de ce projet, nous avons appliqué plusieurs bonnes pratiques essentielles. Premièrement, choisir un TTL adapté à chaque type de donnée : les catégories changent rarement et ont un TTL long de trente minutes, tandis que les commentaires dynamiques ont un TTL court de deux minutes. Deuxièmement, l'invalidation doit être systématique.

TTL adapté : catégories 30min, commentaires 2min

Invalidation systématique lors des écritures

Prefixe saas: pour éviter les collisions

Tests unitaires pour valider le comportement

Monitoring et logs pour suivre l'utilisation

Conclusion et Perspectives

Plusieurs pistes d'amélioration sont envisageables pour renforcer encore notre solution de cache distribuée. Premièrement, nous pourrions mettre en place un cluster Redis avec replication pour garantir la haute disponibilité. Deuxièmement, l'ajout d'un cache LRU local côté application pourrait compléter Redis et réduire encore davantage la latence.

- Cluster Redis avec replication pour haute disponibilité

- Cache LRU local côté application

- Cache warming pour précharger les données populaires

- Redis Streams pour invalidation en temps réel multi-serveur

- Dashboard de monitoring des taux de hit et miss

Merci de votre attention

Questions et Discussion

Presente par

Omar Sarsar & Iliass Hariz

2026

Nous vous remercions pour votre attention. Ce projet a demontre que Redis reduit drastiquement les temps de reponse et la charge sur la base de donnees, tout en maintenant la coherence. Nous esperons que cette presentation vous a donne une vision claire du caching distribue.